

Documentation on the Training and Deployment of an Emergency Vehicle Siren Identification Model on the Infineon PSoC™ Edge E84 AI Kit (KIT_PSE84_AI)

LSA 96'

(Lorenzo Pimentel, Santiago Rivera, Alonso Orbegoso)

June 7, 2026

Project Introduction

This documentation describes the development of a Direction of Arrival (DoA) detection system for emergency vehicle sirens, implemented on the Infineon KIT_PSE84_AI evaluation kit (PSoC™ Edge E84, PSE846GPS2DBZC4A) and trained using the DEEPCRAFT™ Studio embedded machine-learning platform. The solution employs a four-microphone array, consisting of two digital PDM microphones (XENSIV™ IM73D122V01) and two analog MEMS microphones (XENSIV™ IM73A135V01), to classify the acoustic signal's arrival direction across eight 45° angular sectors.

This document is divided into separate steps that may have to be carried out simultaneously; this is intentional. The possibility of compartmentalization is very valuable and, in our experience, immensely useful, given that this is also intended to be a group project. In our case, we found it very useful to use a two-device setup: one computer dedicated to training, which allows for greater training power by training locally on the device rather than in the cloud, and the other dedicated to everything related to embedding the software on the board.

1 Physical Preparations

Having good training data is extremely important for an accurate model; therefore, physical preparations make a big difference. In our case, good examples of preparations were:

1. Maintaining a quiet environment, where noise is under control.
2. Standardizing the data collection (in our case, using tape and recording the exact distances between the sound source and the board).

Other ideas could be arranging an environment with few objects so that sound does not reflect, placing the board on a vibration-damping mount to eliminate sound traveling through the table it stands on, or even devising a way to suspend it in the air to capture 360-degree sound. This part of the project is left to creativity.

Component	Specification
MCU	PSoC™ Edge E84 (PSE846GPS2DBZC4A), Arm® Cortex®-M33 + Cortex®-M55
Microphones	4-microphone array: 2× digital PDM (XENSIV™ IM73D122V01) + 2× analog MEMS (XENSIV™ IM73A135V01)
Channels (DEEPCRAFT™ Studio)	Stereo (2 channels)
Sample Rate	16,000 Hz
Word Length	16-bit PCM
PDM Gain	CY_PDM_PCM_SEL_GAIN_5DB
IDE	Eclipse IDE for ModusToolbox™ v3.6 / v3.8
Toolchain	GCC ARM Embedded v14.2.1

2 Hardware Stack

3 Embedded Software

During the hackathon, the code for the model implementation was provided to us; it may be possible to find this exact code, or something similar, on an online open-source code platform. It is extremely important to first check whether there are compilation errors in the flashing code before starting the training, since its entire purpose is to be embedded on the board. If there are no compilation errors, the embedded-software preparation is complete.

In our case, we modified the provided flashing code to record audio through all four microphones. The two analog MEMS microphones are read through the PSoC™ Edge Autonomous Analog (LPPASS) block via its SAR ADC. This substantially increased the model’s accuracy and the number of possible recording directions. All of this was done while the training was taking place; as mentioned, compartmentalization is of great use in this project. Another recommendation, a matter of taste rather than absolute superiority, is to build and flash using the `modus-shell` command-line console instead of the Eclipse IDE for ModusToolbox™ variant. When dealing with embedded software, the connection to the machine is very important, and adding an extra layer of abstraction can lead to problems.

4 Training

Training was performed using DEEPCRAFT™ Studio. The block-based Graph UX is extremely helpful. One recommendation is to set a band-pass filter around the characteristic frequencies of your siren (in our case, approximately 700–2100 Hz); this makes the training data much more precise, since unwanted noise frequencies are removed. It is also recommended to record as many takes as possible; in our case, our most accurate model used a total of around one hour of training data. When recording takes, it is important to vary the training data, to keep it relatively clean of noise (a 70/30 train/validation split is optimal), and to record one direction at a time in order to minimize movement of the audio source. This process is very tedious; it is, however, the most important part of the project, so sparing time is not an option.

5 Possible Problems and What to Do

1. **Microphone setup not showing in DEEPCRAFT™ Studio:** DEEPCRAFT™ Studio shows only two channels of stereo audio, regardless of the four microphones. If you want to test the microphones, block the others with your fingers (or a more elaborate solution) and tap each microphone individually. If this does not work, then there is a problem with the flashing code.
2. **Unexpected accuracy percentage:** Record more data and train again. If the accuracy is still not sufficient, there might be a problem with the training data. If the problem persists, look for solutions using the DEEPCRAFT™ Studio toolbox.
3. **One label appearing considerably more often than others:** This occurs due to recording-setup interference. Try removing all objects from the surface on which the training is taking place; there should be no way for the frequencies to reflect toward unwanted directions.